

Далее мы начнем использовать более полезный образ, чем hello-world. Мы можем запустить Ubuntu просто с помощью `docker run ubuntu`.

```
$ docker run ubuntu
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
83ee3a23efb7: Pull complete
db98fc6f11f0: Pull complete
f611acd52c6c: Pull complete
Digest: sha256:703218c0465075f4425e58fac086e09e1de5c340b12976ab9eb8ad26615c3715
Status: Downloaded newer image for ubuntu:latest
```

Антиклимакс, так как ничего особенного не произошло. Образ был загружен и запущен, и на этом все закончилось. На самом деле он пытался открыть оболочку, но нам нужно добавить несколько флагов для взаимодействия с ней. `-t` создаст [tty](#).

```
$ docker run -t ubuntu
root@f83969ce2cd1:/#
```

Теперь мы внутри контейнера, и если мы введем `ls` и нажмем enter... ничего не произойдет. Потому что наш терминал не отправляет сообщения в контейнер. Флаг `-i` укажет передавать STDIN в контейнер. Если вы застряли с другим терминалом, вы можете просто остановить контейнер.

```
$ docker run -it ubuntu
root@2eb70ecf5789:/# ls
bin boot dev etc home lib lib32 lib64 libx32 media mnt opt proc root
run sbin srv sys tmp usr var
```

Отлично! Теперь мы знаем как минимум 3 полезных флага. `-i` (интерактивный), `-t` (tty) и `-d` (detached).

Давайте добавим еще несколько и запустим контейнер в фоновом режиме:

```
$ docker run -d -it --name looper ubuntu sh -c 'while true; do date; sleep 1; done'
```

:::tip Кавычки

Если вы используете командную строку (Windows), вы должны использовать двойные кавычки вокруг скрипта, т.е. `docker run -d -it --name looper ubuntu sh -c "while true; do date; sleep 1; done"`. Кавычки или двойные кавычки могут преследовать вас позже в течение курса.

:::

- Первая часть, `docker run -d`. Должна быть знакома к настоящему моменту, запускает контейнер в detached режиме.
- За ней следует `-it`, что является сокращением для `-i` и `-t`. Также знакомо, `-it` позволяет вам взаимодействовать с контейнером с помощью командной строки.
- Поскольку мы запустили контейнер с `--name looper`, теперь мы можем легко сослаться на него.

- Образ — `ubuntu` , и то, что следует за ним, — это команда, переданная контейнеру.

И чтобы проверить, что он работает, запустите `docker container ls`

Давайте следить `-f` за выводом логов с помощью

```
$ docker logs -f looper
Thu Mar  1 15:51:29 UTC 2023
Thu Mar  1 15:51:30 UTC 2023
Thu Mar  1 15:51:31 UTC 2023
...
```

Давайте протестируем приостановку `looper` без выхода или остановки. В другом терминале запустите `docker pause looper` . Обратите внимание, как вывод логов приостановился в первом терминале. Чтобы возобновить, запустите `docker unpause looper` .

Держите логи открытыми и подключитесь к работающему контейнеру из второго терминала, используя `'attach'`:

```
$ docker attach looper
Thu Mar  1 15:54:38 UTC 2023
Thu Mar  1 15:54:39 UTC 2023
...
```

Теперь у вас есть логи процесса (STDOUT), работающие в двух терминалах. Теперь нажмите `control+c` в подключенном окне. Контейнер останавливается, потому что процесс больше не работает.

Если мы хотим подключиться к контейнеру, убедившись, что мы не закрываем его из другого терминала, мы можем указать не подключать STDIN с помощью опции `--no-stdin` . Давайте запустим остановленный контейнер с помощью `docker start looper` и подключимся к нему с помощью `--no-stdin` .

Затем попробуйте `control+c`.

```
$ docker start looper

$ docker attach --no-stdin looper
Thu Mar  1 15:56:11 UTC 2023
Thu Mar  1 15:56:12 UTC 2023
^C
```

Контейнер продолжит работу. `Control+c` теперь только отключает вас от STDOUT.

Запуск процессов внутри контейнера с помощью `docker exec`

Мы часто сталкиваемся с ситуациями, когда нам нужно выполнять команды внутри работающего контейнера. Это можно достичь с помощью команды `docker exec` .

Мы могли бы, например, перечислить все файлы внутри директории контейнера по умолчанию (которая является корневой) следующим образом:

```
$ docker exec looper ls -la
total 56
drwxr-xr-x  1 root root 4096 Mar  6 10:24 .
drwxr-xr-x  1 root root 4096 Mar  6 10:24 ..
-rwxr-xr-x  1 root root    0 Mar  6 10:24 .dockerenv
lrwxrwxrwx  1 root root    7 Feb 27 16:01 bin -> usr/bin
drwxr-xr-x  2 root root 4096 Apr 18  2022 boot
drwxr-xr-x  5 root root  360 Mar  6 10:24 dev
drwxr-xr-x  1 root root 4096 Mar  6 10:24 etc
drwxr-xr-x  2 root root 4096 Apr 18  2022 home
lrwxrwxrwx  1 root root    7 Feb 27 16:01 lib -> usr/lib
drwxr-xr-x  2 root root 4096 Feb 27 16:01 media
drwxr-xr-x  2 root root 4096 Feb 27 16:01 mnt
drwxr-xr-x  2 root root 4096 Feb 27 16:01 opt
dr-xr-xr-x 293 root root    0 Mar  6 10:24 proc
drwx-----  2 root root 4096 Feb 27 16:08 root
drwxr-xr-x  5 root root 4096 Feb 27 16:08 run
lrwxrwxrwx  1 root root    8 Feb 27 16:01/sbin -> usr/sbin
drwxr-xr-x  2 root root 4096 Feb 27 16:01 srv
dr-xr-xr-x 13 root root    0 Mar  6 10:24 sys
drwxrwxrwt  2 root root 4096 Feb 27 16:08 tmp
drwxr-xr-x 11 root root 4096 Feb 27 16:01 usr
drwxr-xr-x 11 root root 4096 Feb 27 16:08 var
```

Мы можем выполнить оболочку Bash в контейнере в интерактивном режиме, а затем запускать любые команды внутри этой сессии Bash:

```
$ docker exec -it looper bash

root@2a49df3ba735:/# ps aux
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
root	1	0.2	0.0	2612	1512	pts/0	Ss+	12:36	0:00	sh -c while true; do date; sleep 1; done
root	64	1.5	0.0	4112	3460	pts/1	Ss	12:36	0:00	bash
root	79	0.0	0.0	2512	584	pts/0	S+	12:36	0:00	sleep 1
root	80	0.0	0.0	5900	2844	pts/1	R+	12:36	0:00	ps aux

Из списка `ps aux` мы видим, что наш процесс `bash` получил PID (идентификатор процесса) 64.

Теперь, когда мы внутри контейнера, он ведет себя так, как вы ожидаете от Ubuntu, и мы можем выйти из контейнера с помощью `exit`, а затем либо убить, либо остановить контейнер.

Наш `looper` не остановится по сигналу SIGTERM, отправленному командой `stop`. Чтобы завершить процесс, `stop` следует за SIGTERM сигналом SIGKILL после периода ожидания. В этом случае проще просто использовать `kill`.

```
$ docker kill looper
$ docker rm looper
```

Запуск двух предыдущих команд в основном эквивалентен запуску `docker rm --force loop`

Давайте запустим другой процесс с `-it` и добавим `--rm`, чтобы удалить его автоматически после завершения. `--rm` гарантирует, что не останется мусорных контейнеров. Это также означает, что `docker start` нельзя использовать для запуска контейнера после его завершения.

```
$ docker run -d --rm -it --name loop-it ubuntu sh -c 'while true; do date; sleep 1; done'
```

Теперь давайте подключимся к контейнеру и нажмем `control+p`, `control+q`, чтобы отключиться от STDOUT.

```
$ docker attach loop-it
```

```
Mon Jan 15 19:50:42 UTC 2018
Mon Jan 15 19:50:43 UTC 2018
^P^Qread escape sequence
```

Вместо этого, если бы мы использовали `ctrl+c`, это отправило бы сигнал `kill`, а затем удалило контейнер, поскольку мы указали `--rm` в команде `docker run`.

Упражнение 1.3

:::info Упражнение 1.3: Секретное сообщение

Теперь, когда мы разогрелись, пришло время попасть внутрь контейнера, пока он работает!

Образ `devopsdockeruh/simple-web-service:ubuntu` запустит контейнер, который выводит логи в файл. Зайдите внутрь работающего контейнера и используйте `tail -f ./text.log`, чтобы следить за логами. Каждые 10 секунд часы будут отправлять вам "секретное сообщение".

Отправьте секретное сообщение и команду(ы) в качестве ответа.

:::

Несоответствие платформы хоста

Если вы работаете с Mac M1/M2, вы, скорее всего, столкнетесь со следующим предупреждением при запуске образа `devopsdockeruh/simple-web-service:ubuntu`:

```
WARNING: The requested image's platform (linux/amd64) does not match the detected
host platform (linux/arm64/v8) and no specific platform was requested
```

Несмотря на это предупреждение, вы можете запустить контейнер. Предупреждение в основном говорит, в чем проблема: образ использует другую архитектуру процессора, чем ваша машина.

Образ можно использовать, потому что Docker Desktop для Mac по умолчанию использует эмулятор, когда архитектура процессора образа не соответствует хостовой. Однако важно отметить, что эмулированное выполнение может быть менее эффективным с точки зрения производительности, чем запуск образа на совместимой нативной архитектуре процессора.

Когда вы запускаете `docker run ubuntu`, например, вы не получаете предупреждение, почему? Довольно много популярных образов являются так называемыми [многоплатформенными образами](#), что означает, что один образ содержит варианты для разных архитектур. Когда вы собираетесь загрузить или запустить такой образ, Docker обнаружит архитектуру хоста и даст вам правильный тип образа.

Ubuntu в контейнере — это просто... Ubuntu

Контейнер, работающий с образом Ubuntu, работает довольно похоже на обычную Ubuntu:

```
$ docker run -it ubuntu
root@881a1d4ecff2:/# ls
bin  dev  home  media  opt   root  sbin  sys  usr
boot etc  lib   mnt    proc  run   srv   tmp  var
root@881a1d4ecff2:/# ps
  PID TTY          TIME CMD
    1 pts/0      00:00:00 bash
   13 pts/0      00:00:00 ps
root@881a1d4ecff2:/# date
Wed Mar  1 12:08:24 UTC 2023
root@881a1d4ecff2:/#
```

Образ, например Ubuntu, уже содержит хороший набор инструментов, но иногда нужного нам инструмента нет в стандартном дистрибутиве. Допустим, мы хотим редактировать некоторые файлы внутри контейнера. Старый добрый редактор [Nano](#) идеально подходит для наших целей. Мы можем установить его в контейнере, используя [apt-get](#):

```
$ docker run -it ubuntu
root@881a1d4ecff2:/# apt-get update
root@881a1d4ecff2:/# apt-get -y install nano
root@881a1d4ecff2:/# cd tmp/
root@881a1d4ecff2:/tmp# nano temp_file.txt
```

Как видно, установка программы или библиотеки в контейнер происходит так же, как установка в "обычной" Ubuntu. Замечательное отличие в том, что установка Nano не является постоянной, то есть, если мы удалим наш контейнер, все исчезнет. Мы скоро увидим, как получить более постоянное решение для создания образов, идеально подходящих для наших целей.

Упражнение 1.4

:::info Упражнение 1.4: Отсутствующие зависимости

Запустите образ Ubuntu с процессом `sh -c 'while true; do echo "Input website:"; read website; echo "Searching.."; sleep 1; curl http://$website; done'`

Если вы на Windows, вы захотите поменять `'` и `"` местами: `sh -c "while true; do echo 'Input website: '; read website; echo 'Searching..'; sleep 1; curl http://$website; done' .`

Вы заметите, что для правильного выполнения отсутствует несколько вещей. Обязательно напомните себе, какие флаги использовать, чтобы контейнер действительно ждал ввода.

Также обратите внимание, что `curl` НЕ установлен в контейнере. Вам придется установить его изнутри контейнера.

Протестируйте ввод `helsinki.fi` в приложение. Оно должно ответить примерно так

```
<html>
  <head>
    <title>301 Moved Permanently</title>
  </head>

  <body>
    <h1>Moved Permanently</h1>
    <p>The document has moved <a href="http://www.helsinki.fi/">here</a>.</p>
  </body>
</html>
```

На этот раз верните команду, которую вы использовали для запуска процесса, и команду(ы), которые вы использовали для исправления возникших проблем.

Подсказка для установки отсутствующих зависимостей вы можете запустить новый процесс с помощью `docker exec` .

- Это упражнение имеет несколько решений, если `curl` для `helsinki.fi` работает, значит, оно выполнено. Можете ли вы придумать другие (умные) решения?

...